Software Architecture and Design Specification

Enterprise Conference Room Booking System

Project: Enterprise Conference Room Booking System (CRB)
Version: 0.1
Authors: **SINCHANA K PES1UG24AM405**
DIRAN SOMANNA PES1UG24AM364
DISHANTH GOWDA H D PES1UG24AM365
PAVAN D C PES1UG24AM383
Date: 15-09-2026

Revision History

Version	Date	Author	Change Summary
0.1	15-09-2026	**SINCHANA K** **DIRAN SOMANNA** **DISHANTH GOWDA** **PAVAN D C**	Initial SAD draft derived from the CRB SRS (v0.1)

Approvals

Role	Name	Signature / Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Enterprise Conference Room Booking System (CRB). It translates the functional, non-functional, and security requirements defined in CRB_SRS.docx into a concrete component structure, technology stack, data model direction, API design, and security architecture.

1.2 Scope

Covers the CRB backend services (authentication/RBAC, room search, booking engine, admin room management, notifications, reporting), the web client, and their integration with the corporate SSO provider and email gateway. Excludes the identity provider's internal implementation and the employee directory's internal implementation - both are treated as external systems accessed via API.

1.3 Audience

Developers, QA engineers, security auditors, instructors, and the four-person team's Dev and Test tracks.

1.4 Definitions

- CRB - Conference Room Booking (system short name)
- RBAC - Role-Based Access Control
- SSO - Single Sign-On
- ADR - Architecture Decision Record
- TTL - Time To Live (booking hold expiry)
- STRIDE - Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

- PII - Personally Identifiable Information

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 are the architectural and design deliverables: UML diagrams (placeholders marked for the team to insert), component responsibilities, the chosen architecture pattern with rationale, the technology stack, API design, and the security architecture (STRIDE). Rationale for each significant choice is kept separately in CRB_Architecture_Decisions.md so this document stays a specification, not a debate.

2.2 Related Documents

- CRB_SRS.docx - functional, non-functional, and security requirements (source of truth for CRB-F-###, CRB-NF-###, CRB-SR-### IDs used throughout this document)
- CRB_Requirements_Catalog.docx - mandatory vs. add-on requirement discussion
- Test_Plan_Template for SE.docx - test strategy, to be filled once this SAD is approved
- CRB_Architecture_Decisions.md - rationale for each architectural and technology choice made in this document

3. Architecture

3.1 Goals & Constraints

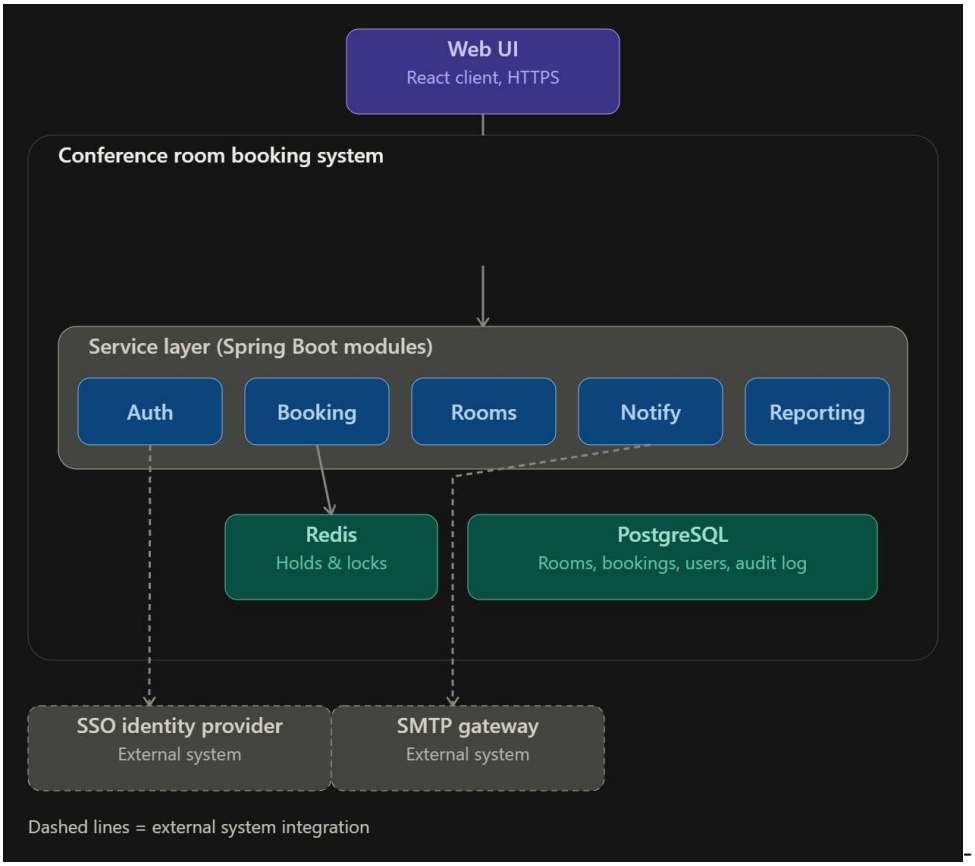
Goals: conflict-free booking, 99.5% business-hours availability (CRB-NF-002), search responses within 2 seconds (CRB-NF-001), 500 concurrent users (CRB-NF-003).

Constraints: must integrate with the corporate SSO/identity provider, must comply with internal PII handling policy, must run within the enterprise network, no dedicated on-site hardware beyond an optional room-display tablet (out of scope for the mandatory feature set).

3.2 Stakeholders & Concerns

- Employees: fast, conflict-free booking; a search experience comparable to Outlook/Google Calendar room booking.
- Facilities Admins: reliable room inventory management, maintenance blocking, utilization visibility.
- IT Security / Auditors: RBAC enforcement, PII protection, attributable admin actions.
- Developers (4-person team): modular boundaries that map cleanly to the four ownership tracks - Auth, Booking Engine, Room/Resource Admin, Notifications/Integration.

3.3 Component (UML) Diagram



- shows Web UI, API Gateway/Auth Filter, Booking Service, Room Admin Service, Notification Service, Reporting Service, Auth Service, PostgreSQL, and Redis, with arrows for: UI to Gateway (HTTPS/REST), Gateway to each service (internal REST), Booking/Admin Service to PostgreSQL, Booking Service to Redis (holds/locks), Notification Service to SMTP gateway, Auth Service to corporate IdP.

3.4 Component Descriptions

Component	Responsibility	Related Requirements
Web UI	Responsive client for search, booking, modification, and the Admin console; renders WCAG 2.1 AA-compliant views.	CRB-F-010/011/012, CRB-NF-005
API Gateway / Auth Filter	Terminates TLS, validates session tokens on every request, and enforces RBAC server-side before a request reaches any service.	CRB-SR-001, CRB-SR-002
Auth Service	Integrates with corporate SSO, issues/validates session tokens, applies account lockout and 30-minute idle timeout.	CRB-F-001/002/003, CRB-SR-004
Booking Service	Owns room search, booking creation with conflict detection, temporary holds, reschedule/cancel, and recurring series logic.	CRB-F-010-012, CRB-F-020-023, CRB-F-030-032, CRB-F-040/041
Room Admin Service	Owns room/equipment inventory CRUD and maintenance-block scheduling.	CRB-F-050, CRB-F-051
Notification Service	Sends confirmation, reminder, and cancellation emails via the corporate SMTP gateway on a scheduled/event basis.	CRB-F-060/061/062
Reporting Service	Aggregates booking data into the room utilization report.	CRB-F-070
PostgreSQL (primary datastore)	Durable storage for rooms, bookings, users/roles, and the admin audit log; PII columns encrypted at rest.	CRB-NF-004, CRB-SR-003, CRB-SR-006
Redis (cache / lock store)	Holds the 2-minute booking-hold TTL and provides a distributed lock used during conflict-check-and-create to prevent race conditions.	CRB-F-020, CRB-F-021, CRB-F-032

3.5 Chosen Architecture Pattern and Rationale

Layered / modular monolith, deployed as a single backend application internally organized into the six services listed in 3.4 as separate modules (package-per-module, not process-per-module). Microservices were considered and rejected for this course scope - see CRB_Architecture_Decisions.md for the full comparison. The module boundaries in 3.4 map directly to the four team ownership tracks, so the pattern can be split into real microservices later without a redesign if utilization ever requires it.

3.6 Technology Stack & Data Stores

- Backend: Java 17, Spring Boot (REST controllers, Spring Security for RBAC, Spring Scheduler for reminder jobs)
- Frontend: React, served as a static SPA behind the API Gateway
- Primary database: PostgreSQL (rooms, bookings, users, roles, audit log)
- Cache / distributed lock: Redis (booking holds, rate limiting)
- Identity: corporate SSO via OAuth2/OpenID Connect (simulated with Keycloak in the course environment)
- Email: corporate SMTP gateway (or a local SMTP test double such as MailHog in development)
- Transport security: TLS 1.2+ terminated at the API Gateway (CRB-SR-001)

3.7 Risks & Mitigations

Risk	Mitigation
Race condition allows two users to double-book the same room (CRB-F-020)	Redis distributed lock around the check-and-create sequence, plus a database-level unique constraint on (room_id, time_range) as a second line of defense
Corporate SSO/IdP unavailable, blocking all logins	Local fallback authentication path with the same lockout policy (CRB-F-003); IdP health check surfaced on the Admin console
Notification job backlog delays reminder emails past the 10-minute window (CRB-F-061)	Scheduled job runs on a short interval (e.g. every minute) and queries only upcoming bookings in the next reminder window, keeping each run's workload small
Recurring-booking series data model becomes a bottleneck for conflict checks at 90-day horizon	Materialize each occurrence as its own booking row at series-creation time rather than computing conflicts on the fly
PII exposure if database backups are mishandled	Column-level encryption at rest (CRB-SR-006) plus access-controlled backup storage

3.8 Traceability to Requirements

Req ID	Requirement (short)	Component
CRB-F-001/002/003	SSO auth, RBAC, account lockout	Auth Service
CRB-F-010/011/012	Room search, filter, availability	Booking Service
CRB-F-020/021/022/023	Booking creation, hold, attendees, confirmation	Booking Service, Redis
CRB-F-030/031/032	Reschedule, cancel, hold auto-release	Booking Service, Redis
CRB-F-040/041	Recurring bookings and occurrence edits	Booking Service
CRB-F-050/051	Room/equipment CRUD, maintenance block	Room Admin Service
CRB-F-060/061/062	Confirmation, reminder, cancellation email	Notification Service
CRB-F-070	Utilization report	Reporting Service

Req ID	Requirement (short)	Component
CRB-NF-001/002/003	Performance, availability, scalability	API Gateway, PostgreSQL, Redis
CRB-SR-001-006	TLS, RBAC enforcement, audit log, session timeout, input validation, PII encryption	API Gateway, Auth Service, PostgreSQL

3.9 Security Architecture

Threat modeling (STRIDE), mapped to the CRB security requirements defined in the SRS:

Threat	Description in CRB Context	Mitigation
Spoofing	An attacker impersonates an employee or admin to book or manage rooms on their behalf.	Corporate SSO authentication (CRB-F-001); session tokens over TLS only
Tampering	A request is modified in transit, or a client-side RBAC check is bypassed by calling the API directly.	TLS 1.2+ (CRB-SR-001); server-side RBAC enforcement on every endpoint (CRB-SR-002)
Repudiation	An admin denies having deactivated a room or cancelled another user's booking.	All admin actions logged with actor ID, timestamp, and detail (CRB-SR-003)
Information disclosure	Attendee names/emails or booking details leak via an unencrypted channel or database export.	TLS in transit (CRB-SR-001); PII encrypted at rest (CRB-SR-006)
Denial of service	A flood of search or booking requests degrades availability for legitimate users.	Rate limiting at the API Gateway; horizontal scalability target of 500 concurrent users (CRB-NF-003)
Elevation of privilege	An Employee-role user calls an Admin-only endpoint directly, bypassing the UI.	Server-side RBAC enforcement rejects the call regardless of UI restrictions (CRB-SR-002)

4. Design

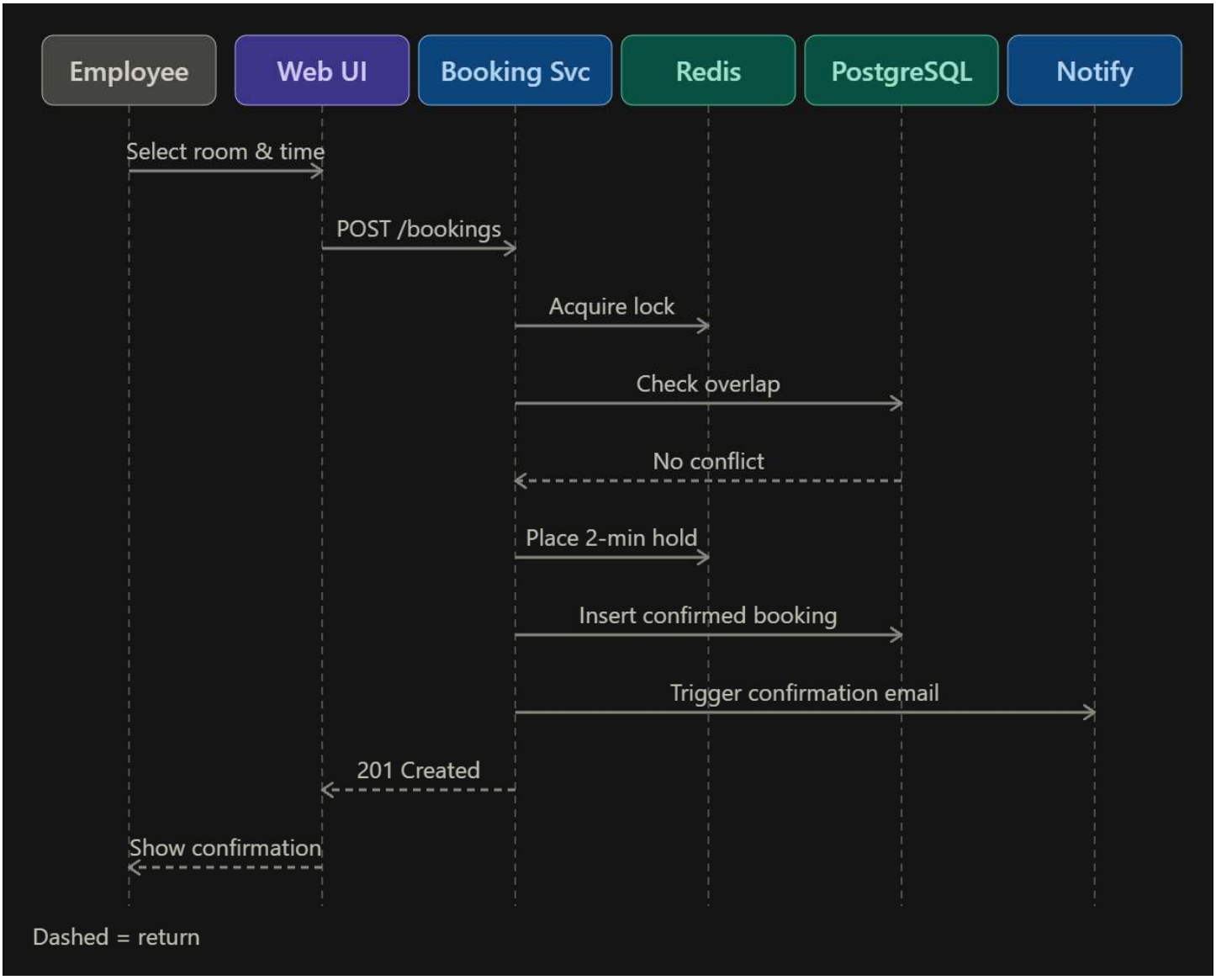
4.1 Design Overview

The system is designed as layered modules (presentation, API/gateway, service, data access) with clear module boundaries matching the four team ownership tracks defined in the SRS, so each track can develop and test its module with minimal cross-module coupling.

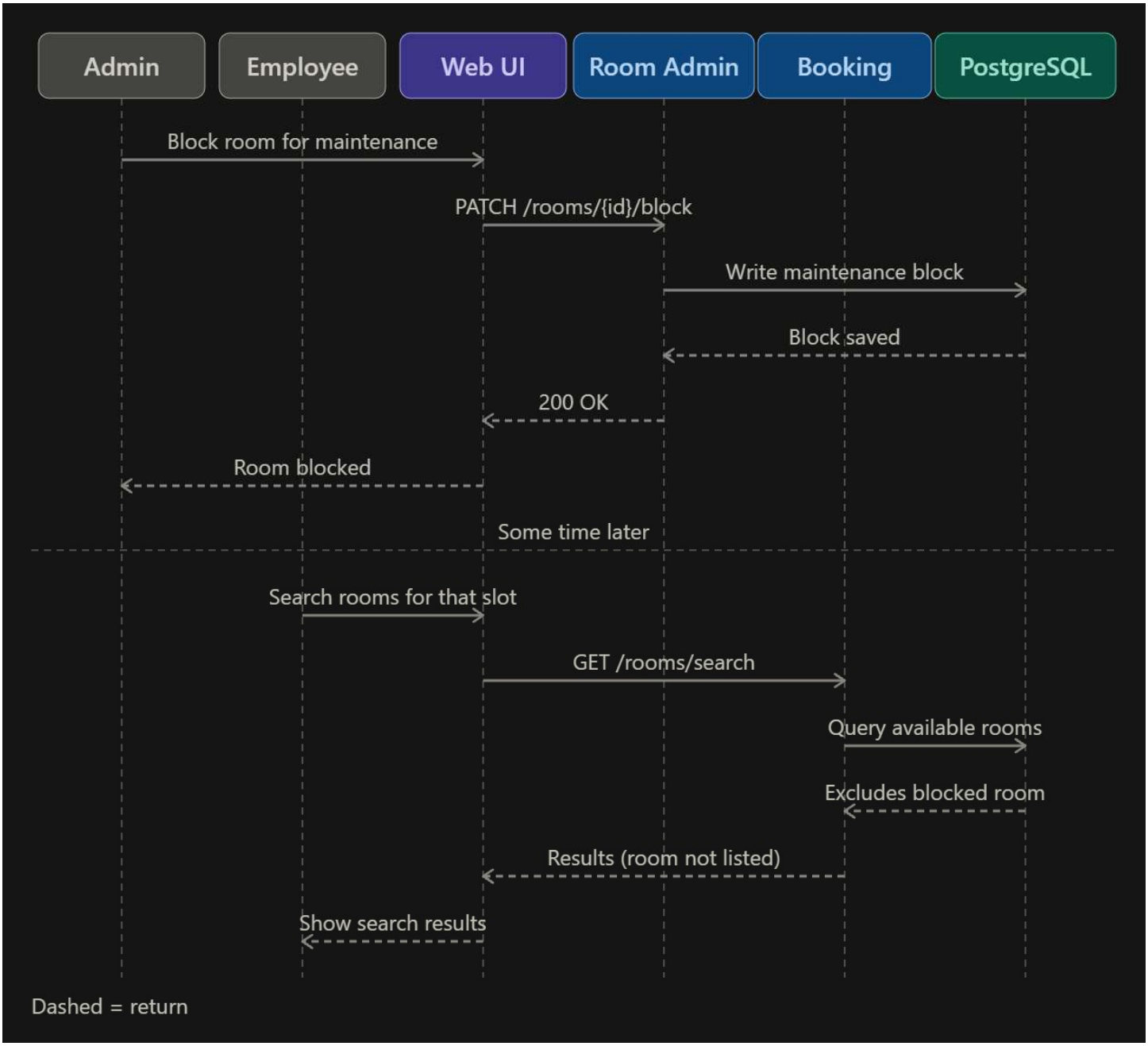
4.2 UML Sequence Diagrams

At least 2 sequence diagrams are required, covering flows specific to this project:

- Flow 1 - Booking creation with conflict detection: Employee -> Web UI -> API Gateway -> Booking Service -> Redis (acquire lock) -> PostgreSQL (check overlap) -> Redis (place 2-min hold) -> Booking Service -> Notification Service (confirmation email) -> Web UI.
- Flow 2 - Room maintenance block preventing a search result: Facilities Admin -> Web UI -> API Gateway -> Room Admin Service -> PostgreSQL (write maintenance block) ... later ... Employee -> Web UI -> API Gateway -> Booking Service -> PostgreSQL (search excludes blocked room) -> Web UI.



Booking creation



maintenance block

4.3 API Design

Interface definitions for three components (exceeds the template's 2-component minimum):

Field	Value
Endpoint	/bookings
Method	POST
Component	Booking Service
Request	{ roomId, startTime, endTime, attendeeEmails[] }
Response	{ bookingId, status, confirmationSentAt }
Errors	401 Not authenticated, 404 Room not found, 409 Time slot conflict

Field	Value
Endpoint	/rooms/search
Method	GET
Component	Booking Service
Request	?location&date&startTime&endTime&minCapacity&equipment[]
Response	[{ roomId, name, capacity, equipment[], location }]
Errors	400 Invalid time range, 401 Not authenticated

Field	Value
Endpoint	/admin/rooms
Method	POST
Component	Room Admin Service
Request	{ name, location, capacity, equipment[] }
Response	{ roomId, status }
Errors	401 Not authenticated, 403 Not an Admin, 400 Invalid room data

4.4 Error Handling, Logging & Monitoring

Standardized error response shape ({ code, message }) across all endpoints; no PII or session tokens in logs. Admin actions are logged separately to the audit table (CRB-SR-003). Monitoring tracks booking conflict-rejection rate, search response time (CRB-NF-001), notification delivery failures, and API availability against the 99.5% business-hours target (CRB-NF-002).

4.5 UX Design

Web UI follows WCAG 2.1 AA guidelines (CRB-NF-005): screen-reader labels on all interactive elements, a high-contrast mode, and keyboard-only navigation support for search, booking, and the Admin console.

4.6 Open Issues & Next Steps

- Final FR/add-on scope is pending team sign-off before the UML diagrams in 3.3 and 4.2 are finalized.
- Add-on candidates such as calendar sync, QR check-in, or IoT occupancy sensors would each require an ADR and a SAD update before implementation - none are committed in this draft.
- Load-testing plan for the 500-concurrent-user target (CRB-NF-003) is not yet defined - to be covered in the Test Plan.

5. Appendices

5.1 Glossary

CRB, RBAC, SSO, ADR, TLS, STRIDE, PII, TTL - see Section 1.4.

5.2 References

- IEEE 42010 (Architecture description)
- OWASP Top 10 / ASVS (application security baseline)
- NIST SP 800-160 (systems security engineering)
- WCAG 2.1 AA (accessibility)

5.3 Tools

- Diagramming: PlantUML, draw.io
- API design/documentation: Swagger / OpenAPI
- Issue tracking: GitHub Projects